

SYSTEM WYPOŻYCZALNI SAMOCHODOWEJ

Wykonawcy:

Michał Wyrembek

Filip Biadała

1. Opis projektu

System obsługi wypożyczalni samochodowej ma na celu usprawnienie zarządzania wynajmem pojazdów poprzez wygodną i intuicyjną aplikację internetową.

Główną funkcjonalnością systemu jest umożliwienie użytkownikom przeglądania dostępnych pojazdów, dokonywania rezerwacji oraz realizacji płatności. Klienci mogą w prosty sposób przeglądać flotę pojazdów, która zawiera szczegółowe opisy, zdjęcia oraz informacje na temat warunków wynajmu i cen. Wynajem pojazdów możliwy jest po zarejestrowaniu konta i zalogowaniu się do systemu, co umożliwia dostęp do dodatkowych funkcji, takich jak podgląd aktywnych rezerwacji, historia transakcji oraz finalizacja wynajmu po jego zakończeniu, włącznie z możliwością wystawienia oceny.

Kalendarz umożliwia użytkownikom łatwe sprawdzenie, które pojazdy są dostępne w wybranych terminach, a każda nowa rezerwacja natychmiast aktualizuje podgląd. Dzięki temu użytkownicy mogą planować wynajem w najdogodniejszym czasie, mając pełną przejrzystość zarówno w kwestii dostępności, jak i cen, co optymalizuje proces rezerwacji i pomaga efektywnie zarządzać flotą.

Użytkownicy mogą otrzymywać przypomnienia o nadchodzących terminach zwrotu pojazdu oraz informację o dokonanych płatnościach i zmianach w rezerwacjach. System posiada także funkcję oceny pojazdów, która pozwala klientom na wystawianie opinii po zakończeniu wynajmu, co wspiera transparentność i buduje zaufanie do wypożyczalni.

Całość systemu jest nadzorowana przez administratora, który ma możliwość zarządzania zarówno użytkownikami, jak i pojazdami oraz monitorowania poprawności działania platformy. Dzięki temu system obsługi wypożyczalni samochodowej zapewnia wygodę zarówno dla użytkowników, jak i operatorów, zwiększając efektywność oraz umożliwiając lepsze zarządzanie flotą pojazdów i transakcjami.

2. Specyfikacja technologii

Backend:

- .NET 8. 11.3.0
 - EntityFramework 11.11.0

Frontend:

- Bootstrap 5.1.0

Baza danych:

- MS SQL

System zarządzania wersjami:

- GitHub

3. Instrukcje uruchomienia projektu

Wymagania wstępne

- 1) Środowisko uruchomieniowe:
 - .NET 8.0 SDK
 - Visual Studio 2022 (z rozszerzeniami: ASP.NET i Entity Framework).
- 2) Baza danych:
 - MS SQL Server 2022
- 3) Narzędzia dodatkowe:
 - Konsola menadżera pakietów NuGet

Konfiguracja projektu

- 1) Sklonowanie repozytorium

git clone <URL>

- 2) Konfiguracja bazy danych (plik appsettings.json)

```
"DefaultConnection":  
"Server=(localdb)\\mssqllocaldb;Database=CarRentalStudio;Tr  
usted_Connection=True;MultipleActiveResultSets=true"
```

- 3) Aktualizacja bazy danych

PM> Update-Database

Uruchomienie projektu

- 1) Otwórz projekt w Visual Studio.
- 2) Ustaw projekt startowy na CarRentalStudio.
- 3) Uruchom aplikację

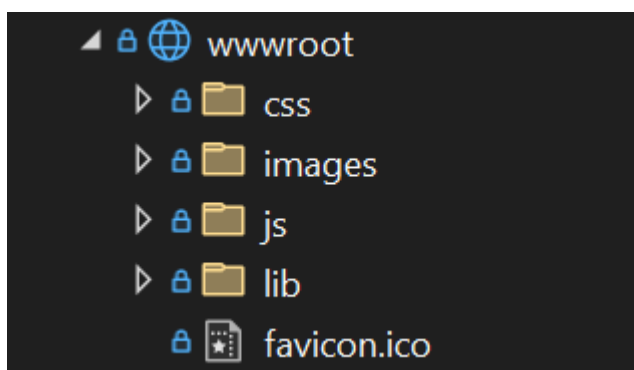
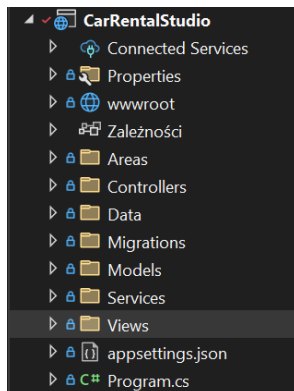
Dostęp do aplikacji

Aplikacja będzie dostępna pod adresem:

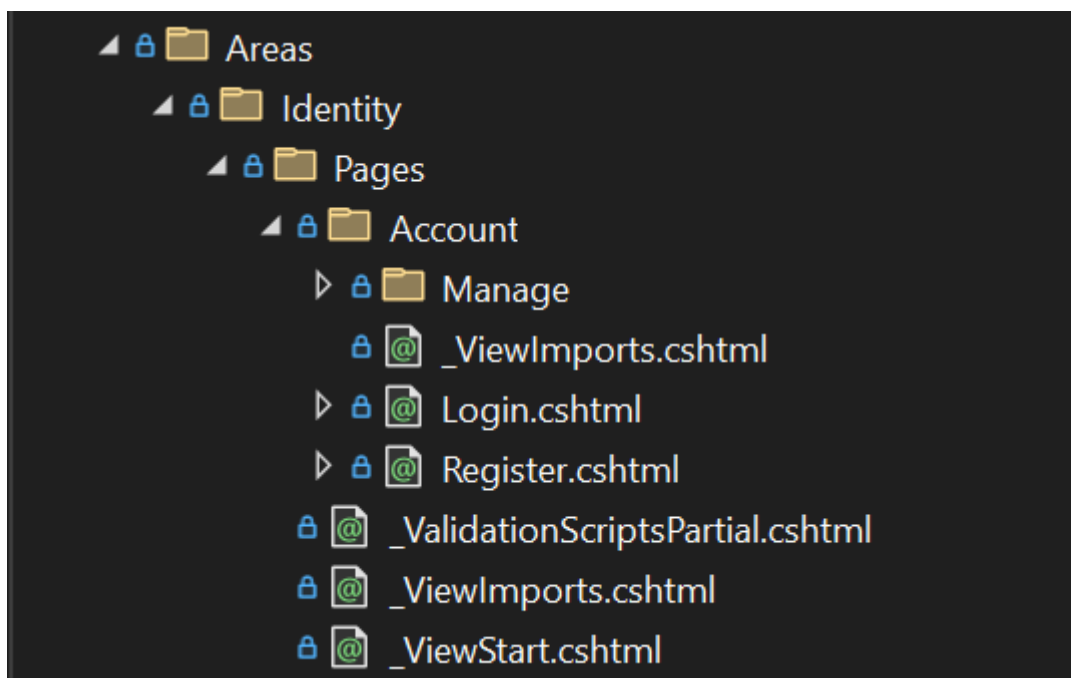
<http://localhost:<port>>

Ręczne uruchomienie nie będzie wymagane, ponieważ aplikacja uruchomi się automatycznie podczas uruchomienia projektu w Visual Studio

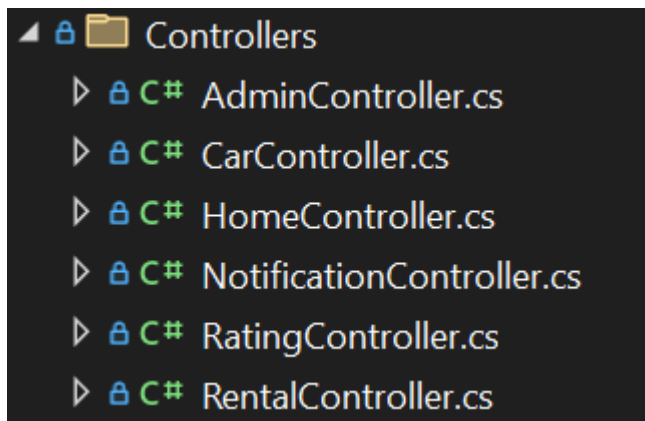
4. Opis struktury projektu



Wwwroot – zasoby statyczne, pliki skryptów, stylów i obrazy



Areas – korzystamy z Identity Framework, więc znajdują się tutaj wszystkie pliki konfiguracyjne dotyczące tego frameworku.

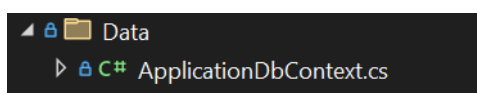


Controllers - zawiera wszystkie wykorzystywane w aplikacji kontrolery. Home wykorzystywany jest do widoku strony głównej, polityki prywatności oraz tworzenia opinii. Notification nie jest obecnie wykorzystywany w naszej aplikacji. Admin, Car, Rating oraz Rental posiadają metody CRUD oraz kilka unikatowych funkcji:

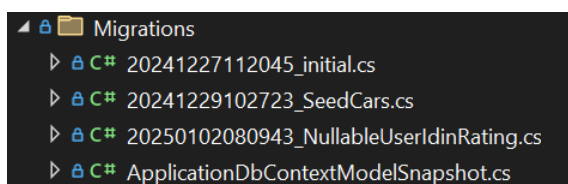
Car -> Wyświetlenie szczegółów samochodu

Car -> Dostępność samochodu

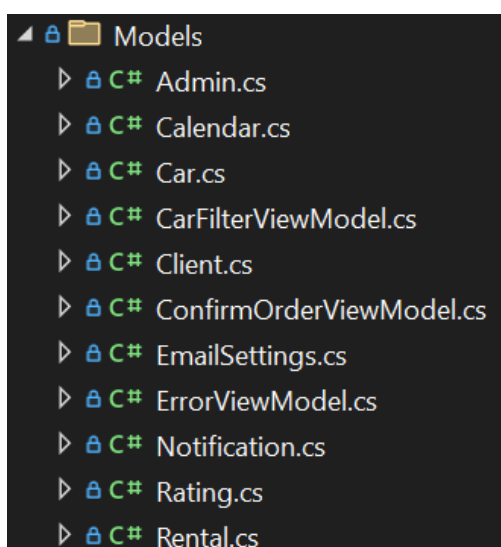
Rental -> Wyświetlenie historii wypożyczeń



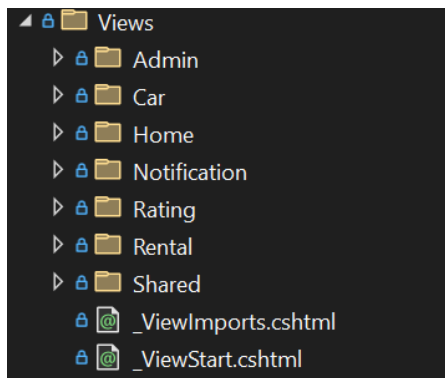
Data – struktura do tworzenia bazy danych. Klasy seedujące dane



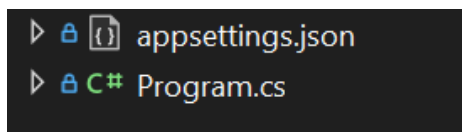
Migrations – wszystkie pliki migracji Entity Framework



Models – katalog modeli wykorzystywanych w projekcie. Zawiera wszystkie klasy wraz z polami i ograniczeniami.



Views – widoki Razor generujące UI



Pliki konfiguracyjne połączenia z bazą danych oraz budowania aplikacji.

Konwencje nazewnictwa

Klasy modeli mają nazwy w liczbie pojedynczej.

Widoki są nazywane zgodnie z akcjami w kontrolerze, np. Details.cshtml odpowiada akcji Details().

5. Wylistowanie modeli

- 1) **Car** – Model używany do tworzenia samochodów. Zawiera wszystkie specjalizacje danego samochodu, takie jak marka model, rok produkcji oraz dane techniczne pojazdu.

i. Pola:

- **Id** [Key]
- **Brand** [Required]
- **Model** [Required]
- **Year** [Required] [Range(1900, int.MaxValue, ErrorMessage = "Rok musi być większy niż 1900.")]
- **Mileage** [Required] [Range(0, float.MaxValue, ErrorMessage = "Przebieg musi być dodatnią liczbą.")]
- **HorsePower** [Required] [Range(0, int.MaxValue, ErrorMessage = "Moc silnika musi być dodatnią liczbą.")]
- **Torque** [Required] [Range(0, int.MaxValue, ErrorMessage = "Moment obrotowy musi być dodatnią liczbą.")]
- **EngineCapacity** [Required] [Range(0.0, float.MaxValue, ErrorMessage = "Wartość musi być liczbą większą od zera.")]
- **FuelType** [Required]
- **Transmission** [Required]
- **BodyType** [Required]
- **Drive** [Required]

- **Acceleration** [Required] [Range(0.0, double.MaxValue, ErrorMessage = "Wartość musi być liczbą większą od zera.")]
- **VMax** [Required] [Range(0.0, double.MaxValue, ErrorMessage = "Wartość musi być liczbą większą od zera.")]
- **DailyRate** [Required] [Range(0.0, double.MaxValue, ErrorMessage = "Wartość musi być liczbą większą od zera.")]
- **IsAvailable**
- **Image**
- **ICollection<Rental> Rentals**

2) **CarFilterViewModel** – widok modelu używany przy filtrowaniu samochodów, który tworzy listę z marek samochodów. Nie posiada z tego powodu ograniczeń.

i. Pola

- **List<string> Brands**
- **List<Car> Cars**
- **MinPrice**
- **Maxprice**

3) **Client** – model klient dziedziczy z IdentityUser, zawiera relację z rentals aby w wypożyczeniu określić użytkownika aplikacji wynajmującego samochodów jako klienta.

i. Pola

- **ICollection<Rental> Rentals**

4) **ConfirmOrderViewModel** – widok modelu do wyświetlania potwierdzenia rezerwacji

i. Pola

- **StartDate**
- **EndDate**
- **RentalDays**
- **TotalPrice**
- **CarId**
- **CarBrand**
- **CarModel**
- **PricePerDay**

5) **Rating** – model przechowujący cechy wszystkich opinii wystawianych przez użytkowników. Opinie zawierają tytuł, opis, ocenę oraz użytkownika, który ją wystawił.

i. Pola:

- **Id**
- **Title**
- **Description**
- **Mark**

- UserId
- IdentityUser? User

6) **Rental** – model, który zawiera wszystkie parametry wykorzystywane przy rezerwacji, takie jak dane samochodu, klienta oraz cenę i datę wynajmu.

i. Pola:

- Id [Key]
- ClientId [ForeignKey("Client"), Required]
- IdentityUser? Client
- CarId [ForeignKey("Car"), Required]
- Car? Car
- RentalStart [Required]
- RentalEnd [Required] [CustomValidation(typeof(Rental), nameof(ValidateRentalPeriod))]
- Price [Required, Range(0, double.MaxValue), Precision(18, 2)]
- IsActive

```

1 Odwołanie
public static ValidationResult? ValidateRentalPeriod(DateTime rentalEnd, ValidationContext context)
{
    var instance = context.ObjectInstance as Rental;
    if (instance != null && rentalEnd <= instance.RentalStart)
    {
        return new ValidationResult("Data zakończenia musi być po dacie rozpoczęcia.");
    }
    return ValidationResult.Success;
}

2 Odwołania: 2
public decimal CalculatePrice()
{
    if (Car == null)
        throw new InvalidOperationException("Samochód nie został przypisany.");
    return (RentalEnd - RentalStart).Days * Car.DailyRate;
}

```

6. Wylistowanie kontrolerów

1) AdminController

i. Metody

- Index()
 - a. Zwrócenie widoku
 - b. `return View();`
- Cars()
 - a. Wyświetlenie listy samochodów
 - b. `return PartialView("_CarList", cars);`
- CreateCar()
 - a. Dodanie samochodu
 - b. `return PartialView("_CarCreate");`
- CreateCar(Car car) [HttpPost]
 - a. Dodanie samochodu
 - b. `return PartialView("_CarCreate", car);`
- EditCar(int id)
 - a. Edycja samochodu
 - b. `return PartialView("_CarEdit", car);`
- EditCar(Car car) [HttpPost]
 - a. Edycja samochodu
 - b. `return PartialView("_CarEdit", car);`
- DeleteCar(int id)
 - a. Usunięcie samochodu
 - b. `return PartialView("_CarList", cars);`
- Users()
 - a. Wyświetlenie listy użytkowników

- b. `return PartialView("_UserList", users);`
- `CreateUser()`
 - a. Dodanie użytkownika
 - b. `return PartialView("_UserCreate");`
- `CreateUser(string email, string password, string role) [HttpPost]`
 - a. Dodanie użytkownika
 - b. `return PartialView("_UserList", users);`
- `EditUser(string id)`
 - a. Edycja użytkownika
 - b. `return PartialView("_UserEdit", user);`
- `EditUser(string id, IdentityUser updatedUser, string role) [HttpPost]`
 - a. Edycja użytkownika
 - b. `return PartialView("_UserEdit", updatedUser);`
`DeleteUser(string id)`
- `DeleteUser(string id)`
 - a. Usunięcie użytkownika
 - b. `return PartialView("_UserList", users);`
- `Rentals()`
 - a. Wyświetlenie listy wypożyczeń
 - b. `return PartialView("_RentalList", rentals);`
- `CreateRental()`
 - a. Dodanie wypożyczenia
 - b. `return PartialView("_RentalCreate");`
- `Create([Bind("Id,ClientId,CarId,RentalStart,RentalEnd,Price")] Rental rental) [HttpPost]`
 - a. Dodanie wypożyczenia
 - b. `return PartialView("_RentalList");`
- `EditRental(int id)`
 - a. Edycja wypożyczenia
 - b. `return PartialView("_RentalEdit", rental);`
- `EditRental(int id, [Bind("Id,ClientId,CarId,RentalStart,RentalEnd,Price")] Rental rental) [HttpPost]`
 - a. Edycja wypożyczenia
 - b. `return PartialView("_RentalEdit", rental);`
- `DeleteRental(int id)`
 - a. Usunięcie wypożyczenia
 - b. `return PartialView("_RentalList", rentals);`

2) CarController

i. Metody:

- `Details(int? id)`
 - a. Wyświetlenie szczegółów samochodu
 - b. `return View(Car);`
- `CarsMainPanel(decimal? minPrice, decimal? maxPrice, List<string> brands)`
 - a. Wyświetlenie głównej listy samochodów w ofercie
 - b. `return View(cars);`
- `Index()`
 - a. `return View(await _context.Cars.ToListAsync());`
- `Create()`
 - a. Dodanie samochodu
 - b. `return View();`
- `Create([Bind("Id, Brand,Model,Year,Mileage,HorsePower,Torque, EngineCapacity, FuelType, Transmission, BodyType, Drive, Acceleration, VMax, DailyRate,Image")] Car car)`
 - a. Dodanie samochodu
 - b. `return View(car);`

- `Edit(int? id)`
 - a. Edycja samochodu
 - b. `return View(car);`
- `Edit(int id, [Bind("Id, Brand, Model, Year, Mileage, HorsePower, Torque, EngineCapacity, FuelType, Transmission, BodyType, Drive, Acceleration, VMax, DailyRate, Image")] Car car)`
 - a. Edycja samochodu
 - b. `return View(car);`
- `Delete(int? id)`
 - a. Usunięcie samochodu
 - b. `return View(car);`
- `DeleteConfirmed(int id)`
 - a. Potwierdzenie usunięcia
 - b. `return RedirectToAction(nameof(Index));`
- `GetUnavailableDates(int carId)`
 - a. Wyciągnięcie niedostępnych dat dla samochodu
 - b. `return Json(unavailableDates);`

3) HomeController

i. Metody:

- `Index()`
 - a. Zwracanie widoku głównego + opinie
 - b. `return View(ratings);`
- `AddRating(Rating rating)`
 - a. Dodawanie opinii
 - b. `return Redirect("Index#formularz-opinii");`
- `HowItWorks()`
 - a. Zwracanie widoku „Jak to działa”, czyli widoku pokazującego działanie wypożyczalni
 - b. `return View();`
- `Privacy()`
 - a. Zwracanie widoku polityki prywatności
 - b. `return View();`
- `Error()`
 - a. Zwracanie błędu
 - b. `return View(new ErrorViewModel { RequestId = Activity.Current?.Id ?? HttpContext.TraceIdentifier });`

4) RatingController

i. Metody:

- `Index()`
 - a. Wyświetlenie widoku głównego
 - b. `return View();`
- `Details(int? id)`
 - a. Wyświetlenie szczegółów opinii
 - b. `return View(rating);`
- `Create()`
 - a. Dodanie opinii (GET)
 - b. `return View();`
- `AddRating(Rating rating)`
 - a. Dodanie opinii (POST)

- b. `return View(rating);`
- `Edit(int? id)`
 - a. Edycja opinii
 - b. `return View(rating);`
- `Edit(int id, [Bind("Id,Title,Description,Mark,UserId")] Rating rating)`
 - a. Edycja opinii
 - b. `return View(rating);`
- `Delete(int? id)`
 - a. Usunięcie opinii
 - b. `return View(rating);`
- `DeleteConfirmed(int id)`
 - a. Potwierdzenie usunięcia
 - b. `return RedirectToAction(nameof(Index));`

5) RentalController

i. Metody:

- `OrderConfirmation(string selectedDates, int carId)`
- `FinalizeOrder(ConfirmOrderViewModel model)`
- `Confirmation()`
- `Index()`
 - a. Wyświetlenie widoku głównego
 - b. `return View(await applicationDbContext.ToListAsync());`
- `Details(int? id)`
 - a. Wyświetlenie szczegółów
 - b. `return View(rental);`
- `Create(int clientId, int carId)`
 - a. Dodanie wypożyczenia
 - b. `return View();`
- `Create([Bind("Id,ClientId,CarId,RentalStart,RentalEnd,Price")] Rental rental)`
 - a. Dodanie wypożyczenia (POST)
 - b. `return View(rental);`
- `Edit(int? id)`
 - a. Edycja wypożyczenia
 - b. `return View(rental);`
- `Edit(int id, [Bind("Id,ClientId,CarId,RentalStart,RentalEnd,Price")] Rental rental)`
 - a. Edycja wypożyczenia
 - b. `return View(rental);`
- `Delete(int? id)`
 - a. Usunięcie wypożyczenia
 - b. `return View(rental);`
- `DeleteConfirmed(int id)`
 - a. Potwierdzenie usunięcia
 - b. `return View(rentals);`
- `RentalHistory()`

- a. Historia wypożyczeń dla każdego użytkownika
- b. `return View(rentals);`

7. Opis systemu użytkowników

1) Role

- User
- Manager
- Admin

Jeśli chodzi o rolę to podczas rejestracji przypisywana jest domyślnie rola User. Administrator ma możliwość zarządzania użytkownikami, więc może zmienić dowolnemu użytkownikowi rolę lub stworzyć nowego użytkownika z odpowiednią rolą.

2) Możliwości

Będąc niezalogowanym, gość, może jedynie przeczytać co jest na stronie, przejrzeć samochody i szczegóły o nich. Dopiero po zalogowaniu/rejestracji, użytkownik ma dostęp do funkcjonalności takich jak rezerwacja pojazdu lub wystawienie opinii oraz do zarządzania swoim kontem. Informację jakie są powiązane z użytkownikiem to rezerwacje. Każdy użytkownik może przejrzeć historię swoich rezerwacji nie mając dostępu do historii rezerwacji innych użytkowników. Reszta informacji dostępnych na stronie jest globalna i nie potrzeba konta, żeby je przeczytać. Wszystkie ukryte zasoby dostępne są tylko dla Administratora.

8. Najciekawsze funkcjonalności

- Sprawdzanie dostępności samochodu na podstawie wcześniej dokonanych wypożyczeń;
- Filtrowanie samochodów po cenie i marce;
- Sortowanie po cenie;
- Panel użytkownika z opcjami zarządzania kontem oraz historią wypożyczeń;
- Podział użytkowników na rolę; (User, Manager, Admin)
- Panel administratora dostępny dla menadżerów oraz admina. Administrator posiada dostęp do zarządzania samochodami, użytkownikami oraz wypożyczeniami, a menadżer może zarządzać tylko wypożyczeniami;
- Dodawanie i przeglądanie opinii klientów;

9. Dodatkowe informacje

Dane logowania podstawowych użytkowników

```
admin_email = admin@admin.com
admin_password = Test123$
manager1_email = michal@manager.com
manager1_password = Michal123$
manager2_email = filip@manager.com
manager2_password = Filip123$
```

https://cylindersi.pl/wp-content/uploads/2024/07/teslaX-6-of-16_1.jpg

Przykładowe dodanie samochodu

Zaloguj się

Email
admin@admin.com

Hasło
Test123\$

☐ Remember me?

Zaloguj się

CarRentalStudio

Home

Jak to działa?

Samochody

Panel Administratora

Menu

Zarządzaj samochodami

Zarządzaj użytkownikami

Zarządzaj wypożyczeniami

Dodaj

Id		Marka	Model
Dodaj samochód			
Marka			
Tesla			
Model			
X			
Rok produkcji			
2024			
Przebieg			
1000			
Moc Silnika			
100			
Moment obrotowy			
100			
Pojemność			
3			
Rodzaj paliwa			
Elektryczny			
Skrzynia biegów			
Automatyczna			
BodyType			
SUV			
Napęd			
RWD			
Przyspieszenie			
3			
Prędkość maksymalna			
250			
Dzienna stawka			
650			
Zdjęcie			
https://cylindersi.pl/wp-content/uploads/2024/07/teslaX-6-of-16_1.jpg			
Dodaj			
11	Tesla	X	2024
		Edytuj	Usuń



Tesla X

od 650,00 zł

 3 s do 100km/h

 Automatyczna

 100 KM / 100 NM

 Sedan

[Zobacz szczegóły](#)

 [Samochod](#)

Tesla X

od 650,00 zł

 3 s do 100km/h

 Automatyczna

 100 KM / 100 NM

 Sedan

[Zobacz szczegóły](#)